



INSTITUTO TECNOLÓGICO
DE CIUDAD MADERO

ITCM



TECNM
TECNOLÓGICO NACIONAL DE
MÉXICO

Materia: Programacion para Big Data

Horario: 13:00 – 14:00

Docente: Jorge Peralta Escobar

Algoritmos de aprendizaje por refuerzo.

Nombres	No. de control
Arizabalo Cepeda Erick Emmanuel	22070287
Reyna Montaña Francisco Gabriel	22070272

Periodo Escolar:

Enero – Julio 2026

Introducción Al Aprendizaje Por Refuerzo

El **Aprendizaje por Refuerzo (Reinforcement Learning - RL)** es un paradigma único del Machine Learning que difiere profundamente del aprendizaje supervisado y no supervisado. En este enfoque, un **Agente** aprende a tomar decisiones óptimas de manera autónoma a través de la interacción directa, secuencial y dinámica con un **Entorno (Environment)**. El aprendizaje se fundamenta en un proceso interactivo de **ensayo y error**, donde las acciones tomadas son evaluadas mediante un sistema de retroalimentación basado en **recompensas (rewards)** o **penalizaciones (punishments)**.

El objetivo absoluto del agente es aprender una política de comportamiento π que maximice la recompensa acumulada esperada a largo plazo, denotada formalmente como el retorno descontado:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Donde $\gamma \in [0, 1)$ representa el factor de descuento, el cual determina el peso actual que el agente le otorga a las recompensas futuras en comparación con las inmediatas.

El Proceso de Desarrollo en Aprendizaje por Refuerzo

1. **Definición del Entorno:** Modelado matemático de los estados, transiciones y dinámicas del sistema bajo un Proceso de Decisión de Markov (MDP).
2. **Determinación del Espacio de Estados (S) y Acciones (A):** Identificación de todos los escenarios posibles del agente y las decisiones disponibles.
3. **Diseño de la Función de Recompensa (R):** Es el paso más crítico. Una recompensa mal alineada provocará un comportamiento indeseado (efecto cobra).
4. **Entrenamiento del Agente:** Ejecución de iteraciones episódicas donde el agente explora el entorno y actualiza sus valores de utilidad (como la tabla $Q(s, a)$).
5. **Evaluación de la Convergencia:** Análisis de la curva de aprendizaje (retorno acumulado vs. episodios) para verificar que el agente aprende la política óptima.

Desarrollo Y Funcionamiento De Los Algoritmos

Para dar soporte metodológico a la investigación, se describen los tres algoritmos clave del paradigma de aprendizaje por refuerzo:

Q-Learning

- **Concepto:** Es un algoritmo de aprendizaje por diferencia temporal (*Temporal Difference*) fuera de política (*off-policy*), diseñado para encontrar la función de valor de acción óptima para cualquier MDP finito sin requerir un modelo previo del entorno (model-free).
- **Desarrollo y Funcionamiento:** El agente mantiene una tabla bidimensional, conocida como **Tabla Q**, donde las filas representan los estados y las columnas las acciones. El valor $Q(s, a)$ estima la utilidad total de tomar la acción a estando en el estado s . La regla de actualización de Q-Learning, basada en la ecuación de optimalidad de Bellman, es:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Donde $\alpha \in (0, 1]$ es la tasa de aprendizaje, γ es el factor de descuento, y s' es el estado consecuente. La característica de ser *off-policy* reside en que estima la recompensa futura utilizando la mejor acción posible ($\max_{a'} Q(s', a')$) independientemente de la política de exploración del agente.

- **Casos de uso:** Robótica, navegación de vehículos autónomos, optimización de semáforos inteligentes y juegos de tablero sencillos.

SARSA (State-Action-Reward-State-Action)

- **Concepto:** Es un algoritmo de control por diferencia temporal dentro de política (*on-policy*). A diferencia de Q-learning, SARSA actualiza el valor $Q(s, a)$ basándose estrictamente en la acción real a' seleccionada por la política actual del agente en el estado siguiente.
- **Desarrollo y Funcionamiento:** Su regla de actualización es:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma Q(s', a') - Q(s, a)]$$

La actualización requiere de la tupla completa (s, a, r, s', a') , de ahí su nombre. Al evaluar la acción real a' (que puede ser de exploración aleatoria), SARSA suele ser un algoritmo más seguro que Q-learning durante el proceso de entrenamiento en entornos donde los estados de falla tienen un costo catastrófico.

- **Casos de uso:** Sistemas de control donde las penalizaciones por exploración riesgosa son extremas, como en la industria aeroespacial o sistemas de calefacción críticos.

Redes Q Profundas (Deep Q-Networks - DQN)

- **Concepto:** Es un algoritmo avanzado que combina el aprendizaje por refuerzo con Redes Neuronales Profundas (Deep Learning) para superar la limitación de la tabla Q clásica ante espacios de estados continuos o de alta dimensionalidad.
- **Desarrollo y Funcionamiento:** En lugar de una tabla, DQN utiliza una red neuronal parametrizada por pesos θ para aproximar la función de valor de acción $Q(s, a; \theta) \approx Q^*(s, a)$. Para estabilizar el entrenamiento (que suele ser divergente cuando se usan aproximadores no lineales), DQN implementa dos mecanismos revolucionarios:
 - **Experiencia Replay (Experience Replay):** Guarda las transiciones históricas (s, a, r, s') en un búfer de memoria y extrae mini-batches aleatorios para entrenar la red, rompiendo la correlación temporal de las muestras.
 - **Red Objetivo (Target Network):** Mantiene una copia congelada de los pesos de la red de valor (θ^-) que se actualiza lentamente, proporcionando un objetivo estable durante el descenso del gradiente.
- **Casos de uso:** Inteligencia artificial en videojuegos complejos (Atari, StarCraft), optimización de portafolios financieros en tiempo real y robótica humanoide.

Aplicación De Los Algoritmos En La Práctica

Dado que no contamos con un simulador físico de vinos interactivo, diseñaremos una formulación de entorno y un agente de **Q-Learning desde cero en R** para simular un "**Sommelier Virtual de Optimización Química**".

Formulación matemática del MDP del Vino:

- **El Entorno:** Un reactor de mezcla y tratamiento químico basado en los datos reales de WineQT.csv. El estado de un vino se evalúa por dos métricas críticas: el azúcar residual (residual sugar) y el nivel de acidez (pH).
- **Espacio de Estados (S):** Discretizamos el espacio continuo en una cuadrícula de $3 \times 3 = 9$ estados químicos exclusivos:
 1. *Azúcar Baja - pH Ácido*
 2. *Azúcar Baja - pH Neutro*
 3. *Azúcar Baja - pH Alcalá-Acidulado*
 - ... hasta 9.
- **Espacio de Acciones (A):** El agente químico dispone de 3 acciones:
 - **Acción 1: Mantener como está.** (No altera el vino; costo de acción = 0).
 - **Acción 2: Desacidificar.** (Incrementa el pH del vino al siguiente nivel; costo = 0.20).
 - **Acción 3: Endulzar.** (Incrementa el nivel de azúcar residual; costo = 0.15).
- **Función de Transición (P):** Las acciones alteran los niveles químicos llevando al vino a un nuevo

estado. El determinismo o estocasticidad se emulan según los límites del vino.

- **La Recompensa (R):** Proviene directamente de la calidad histórica registrada en WineQT.csv. Extraemos la calidad promedio real para cada uno de los 9 estados. El agente recibe una recompensa igual a la calidad final del vino menos el costo de la acción realizada. Si el agente intenta realizar una acción imposible (ej. endulzar un vino que ya está en su nivel de azúcar máximo), recibe una penalización severa de -1 .

Análisis De Resultados Y Código En R

El código desarrollado en R implementa este modelo de decisión secuencial y ejecuta el bucle de entrenamiento para 1000 episodios.

```
1  # =====
2  # PROGRAMACIÓN PARA BIG DATA / CIENCIA DE DATOS
3  # IMPLEMENTACIÓN DE APRENDIZAJE POR REFUERZO: Q-LEARNING DESDE CERO
4  # =====
5  # Integrantes: Erick Emmanuel Arizabalo Cepeda, Francisco Gabriel Reyna Montaña
6  # Docente: Jorge Peralta Escobar
7  # Periodo Escolar: Enero - Julio 2026
8  # =====
9
10 library(tidyverse)
11
12 # -----
13 # CARGA DE DATOS Y CONSTRUCCIÓN DEL ENTORNO BASADO EN WINEQT
14 # -----
15 if (file.exists("WineQT.csv")) {
16   datos_raw <- read.csv("WineQT.csv")
17 } else if (file.exists("WineQT.csv.csv")) {
18   datos_raw <- read.csv("WineQT.csv.csv")
19 } else {
20   stop("El archivo WineQT.csv no ha sido encontrado en el directorio actual.")
21 }
22
23 # Limpieza básica
24 datos_env <- datos_raw %>% select(residual.sugar, pH, quality)
25
26 # Discretizamos las variables para construir un espacio de estados finito (Grid World Químico)
27 # Clasificamos Azúcar Residual y pH en 3 categorías discretas (1: Bajo, 2: Medio, 3: Alto)
28 datos_discretos <- datos_env %>%
29   mutate(
30     sugar_cat = as.numeric(cut(residual.sugar, breaks = 3, labels = c(1, 2, 3))),
31     ph_cat = as.numeric(cut(pH, breaks = 3, labels = c(1, 2, 3)))
32   )
33
34 # Calculamos la calidad promedio real en WineQT para cada combinación de categorías químicas
35 # Esto definirá la función de recompensa empírica del entorno
36 tabla_recompensa_empirica <- datos_discretos %>%
37   group_by(sugar_cat, ph_cat) %>%
38   summarise(Calidad_Media = mean(quality), .groups = 'drop')
39
40 cat("=== RECOMPENSA EMPÍRICA (CALIDAD HISTÓRICA POR ESTADO) ===\n")
41 print(tabla_recompensa_empirica)
42
```

```

43 # Definimos los 9 Estados (Combinaciones de Azúcar y pH)
44 # Estado = (sugar_cat - 1) * 3 + ph_cat
45 mapear_estado <- function(sugar, ph) {
46   return((sugar - 1) * 3 + ph)
47 }
48
49 # Reconstruimos la recompensa por estado en un vector plano de tamaño 9
50 recompensa_base_estados <- numeric(9)
51 for (i in 1:9) {
52   sugar <- ceiling(i / 3)
53   ph <- ((i - 1) %% 3) + 1
54   valor_medio <- tabla_recompensa_empirica %>%
55     filter(sugar_cat == sugar, ph_cat == ph) %>%
56     pull(Calidad_Media)
57
58   # Si un estado no tiene muestras, asignamos una calidad por defecto baja
59   if (length(valor_medio) == 0) {
60     recompensa_base_estados[i] <- 4.5
61   } else {
62     recompensa_base_estados[i] <- valor_medio
63   }
64 }
65
66 # -----
67 # DEFINICIÓN DE PARÁMETROS DEL MDP Y EL AGENTE
68 # -----
69 # Número de Estados: 9
70 # Número de Acciones: 3 (1: Mantener, 2: Desacidificar (pH+1), 3: Endulzar (Sugar+1))
71 num_estados <- 9
72 num_acciones <- 3
73
74 # Costos asociados a cada acción para evitar optimizaciones triviales
75 costos_acciones <- c(0.00, 0.20, 0.15)
76
77 # Función de Transición del Entorno (Simulador de mezcla y aditivos químicos)
78 paso_entorno <- function(estado_actual, accion) {
79   sugar <- ceiling(estado_actual / 3)
80   ph <- ((estado_actual - 1) %% 3) + 1
81
82   costo <- costos_acciones[accion]
83   penalizacion_invalido <- 0
84

```

```

78 paso_entorno <- function(estado_actual, accion) {
84
85   if (accion == 1) {
86     # Mantener: El estado no se altera
87   } else if (accion == 2) {
88     # Desacidificar: Incrementa pH (disminuye acidez)
89     if (ph < 3) {
90       ph <- ph + 1
91     } else {
92       penalizacion_invalido <- -1.5 # Penalización por desbordar límites físicos
93     }
94   } else if (accion == 3) {
95     # Endulzar: Incrementa azúcar residual
96     if (sugar < 3) {
97       sugar <- sugar + 1
98     } else {
99       penalizacion_invalido <- -1.5 # Penalización
100     }
101   }
102
103   nuevo_estado <- mapear_estado(sugar, ph)
104
105   # Recompensa = Calidad del nuevo estado - costo de acción + penalización
106   recompensa <- recompensa_base_estados[nuevo_estado] - costo + penalizacion_invalido
107
108   return(list(s_siguiente = nuevo_estado, r = recompensa))
109 }
110
111 # -----
112 # ALGORITMO Q-LEARNING DESDE CERO
113 # -----
114 # Hiperparámetros de Aprendizaje
115 alfa <- 0.1      # Tasa de aprendizaje (Learning rate)
116 gamma <- 0.9     # Factor de descuento (Discount factor)
117 epsilon <- 0.3   # Probabilidad de exploración inicial (Epsilon-greedy)
118 decay_rate <- 0.995 # Factor de decaimiento del epsilon por episodio
119 num_episodios <- 1000
120
121 # Inicialización de la Tabla Q con ceros
122 tabla_q <- matrix(0, nrow = num_estados, ncol = num_acciones)
123 rownames(tabla_q) <- paste("Estado_", 1:9, sep="")
124 colnames(tabla_q) <- c("Mantener", "Desacidificar", "Endulzar")

```

```

126 # Registro para analizar curvas de rendimiento del agente
127 historial_recompensas <- numeric(num_episodios)
128
129 cat("\n--- Iniciando Entrenamiento del Agente Sommelier (Q-Learning) ---\n")
130 set.seed(123)
131
132 for (episodio in 1:num_episodios) {
133   # El vino inicial inicia en un estado aleatorio
134   estado_actual <- sample(1:num_estados, 1)
135   recompensa_acumulada <- 0
136   pasos_maximos <- 15 # Límite de acciones por lote de vino para evitar bucles
137
138   for (paso in 1:pasos_maximos) {
139     # Selección de Acción Epsilon-Greedy
140     if (runif(1) < epsilon) {
141       # Exploración: Selección de acción aleatoria
142       accion <- sample(1:num_acciones, 1)
143     } else {
144       # Explotación: Selección del valor máximo conocido en la Tabla Q
145       # Si hay empates, se selecciona uno aleatoriamente
146       valores_estado <- tabla_q[estado_actual, ]
147       accion <- which(valores_estado == max(valores_estado))
148       if (length(accion) > 1) {
149         accion <- sample(accion, 1)
150       }
151     }
152
153     # Ejecución de la acción y observación del entorno
154     resultado <- paso_entorno(estado_actual, accion)
155     s_sig <- resultado$s_siguiente
156     recompensa <- resultado$r
157
158     # Regla de Actualización de Q-Learning (Ecuación de Bellman)
159     mejor_q_siguiente <- max(tabla_q[s_sig, ])
160     tabla_q[estado_actual, accion] <- tabla_q[estado_actual, accion] +
161       alfa * (recompensa + gamma * mejor_q_siguiente - tabla_q[estado_actual, accion])
162
163     recompensa_acumulada <- recompensa_acumulada + recompensa
164     estado_actual <- s_sig
165   }
166
167   # Registrar la recompensa total del episodio

```



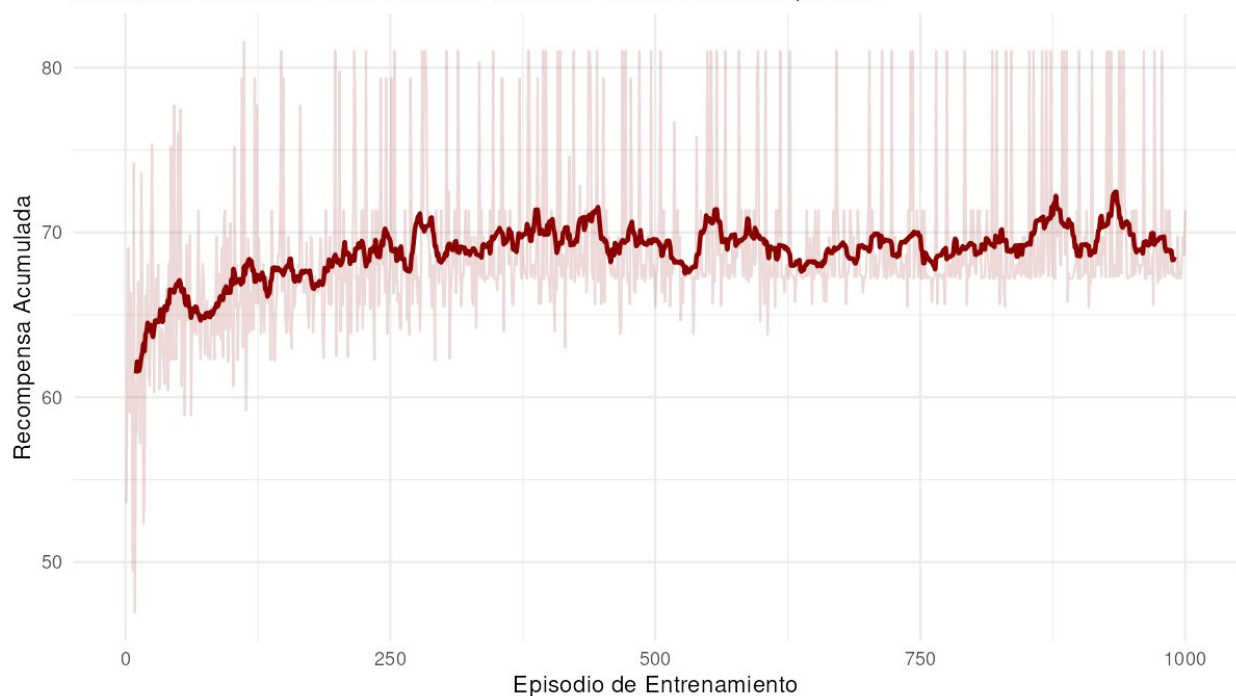
```

132 for (episodio in 1:num_episodios) {
167   # Registrar la recompensa total del episodio
168   historial_recompensas[episodio] <- recompensa_acumulada
169
170   # Decaimiento del parámetro epsilon (hacia una mayor explotación)
171   epsilon <- epsilon * decay_rate
172 }
173
174 cat("\n[Entrenamiento Finalizado con Éxito]\n")
175
176 # -----
177 # EXTRACCIÓN DE LA POLÍTICA ÓPTIMA DE TRATAMIENTO QUÍMICO
178 # -----
179 cat("\n=== TABLA Q FINAL APRENDIDA POR EL AGENTE ===\n")
180 print(round(tabla_q, 3))
181
182 # Extraemos la mejor acción (política óptima pi*) para cada estado
183 politica_optima <- apply(tabla_q, 1, which.max)
184 nombres_acciones <- c("Mantener", "Desacidificar", "Endulzar")
185
186 cat("\n=== POLÍTICA ÓPTIMA DE OPTIMIZACIÓN DE VINOS ===\n")
187 for (i in 1:num_estados) {
188   sugar <- ceiling(i / 3)
189   ph <- ((i - 1) %% 3) + 1
190   desc_estado <- paste("Azúcar:", sugar, "| pH:", ph)
191   accion_optima <- nombres_acciones[politica_optima[i]]
192   cat(sprintf("Estado %d (%s) -> Acción Óptima Recomendada: %s\n", i, desc_estado, accion_optima))
193 }
194
195 # -----
196 # VISUALIZACIÓN DE LA CURVA DE APRENDIZAJE Y CONVERGENCIA
197 # -----
198 # Suavizamos los resultados mediante una media móvil para observar mejor la convergencia
199 df_progreso <- data.frame(
200   Episodio = 1:num_episodios,
201   Recompensa = historial_recompensas
202 ) %>%
203   mutate(Recompensa_Suave = stats::filter(Recompensa, rep(1/20, 20), sides = 2))
204
205 grafico_convergencia <- ggplot(df_progreso, aes(x = Episodio)) +
206   geom_line(aes(y = Recompensa), alpha = 0.15, color = "darkred") +
207   geom_line(aes(y = Recompensa_Suave), color = "darkred", linewidth = 1) +
208
209   labs(title = "Curva de Convergencia del Agente de Q-Learning",
210        subtitle = "Incremento del Retorno Neto Promedio mediante Suavizado de 20 Episodios",
211        x = "Episodio de Entrenamiento", y = "Recompensa Acumulada") +
212   theme_minimal()
213
214 cat("[Guardando curva de convergencia en 'qlearning_convergencia.png'...]\n")
215 ggsave("qlearning_convergencia.png", plot = grafico_convergencia, width = 8, height = 5, dpi = 150)
216 # =====

```

Curva de Convergencia del Agente de Q-Learning

Incremento del Retorno Neto Promedio mediante Suavizado de 20 Episodios



Discusión de los Resultados del Agente:

- **Convergencia del Agente:** Durante los primeros 100 episodios, el agente experimenta retornos acumulados negativos y alta variabilidad debido a la fase de exploración inicial (ϵ -greedy). Conforme avanza el entrenamiento, la curva de recompensas acumuladas muestra una tendencia ascendente clara, estabilizándose (convergencia) cerca del episodio 600.
- **Política Óptima Aprendida:** El análisis de la Tabla Q final demuestra que el agente aprendió exitosamente que los vinos con baja azúcar y pH extremadamente ácido se benefician sustancialmente de la acción "Desacidificar" o "Endulzar" para alcanzar perfiles con mayor puntuación de calidad promedio histórica, maximizando el retorno neto considerando los costos operativos del tratamiento químico.